Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 2

Técnicas de Búsqueda basadas en Poblaciones
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **2**
2.1. Representación de soluciones 2
2.2. Generación aleatoria de soluciones 3
2.3. Operador de reparación (`fix`) 3
2.4. Operador de selección: torneo binario 3
2.5. Operadores de cruce 3
2.6. Operador de mutación 4

3. Algoritmos Genéticos **4**
3.1. Algoritmo Genético Generacional (AGG) 4
3.2. Algoritmo Genético Estacionario (AGE) 5

4. Estructura del Código y Manual de Usuario **5**
4.1. Estructura de directorios 5
4.2. Compilación 6
4.3. Ejecución 6

5. Experimentos y Resultados **7**
5.1. Casos del problema 7
5.2. Resultados 7
5.3. Boxplots 8

6. Análisis de Resultados **10**
6.1. AGG-Arit vs AGG-BLX 10
6.2. AGE-Arit vs AGE-BLX 10
6.3. AGG vs AGE (comparación de esquemas) 10
6.4. AGGs/AGEs vs BL y Greedy 10
6.5. Variabilidad y coste computacional 11

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como el ratio de mejora relativa del valor:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos empleados cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación de generalización.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$, la calidad de la cartera se mide como:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

donde:

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y λ el factor de penalización del riesgo definido en las transparencias del Seminario 2.

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i \in 1 \ldots N$$

Los rangos $[lo, hi]$ varían según el conjunto de datos (véase Sección 5).

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución queda representada como un vector $w \in \mathbb{R}^N$ donde cada componente w_i indica la fracción del capital invertida en el activo i. La restricción de cardinalidad implica que w_i puede ser exactamente cero (no se invierte en esa empresa) o estar en el intervalo $[lo, hi]$.

2.2. Generación aleatoria de soluciones

La generación de la solución inicial es la misma que en la Práctica 1: se elige un subconjunto aleatorio de empresas activas, se les asigna un peso aleatorio en $[lo, hi]$, se normaliza y se aplica el operador de reparación.

2.3. Operador de reparación (`fix`)

El operador de reparación es el mismo que el empleado en la Práctica 1 (tres fases: normalización, recorte de rango, redistribución iterativa). Es especialmente importante en los algoritmos genéticos ya que los operadores de cruce pueden generar soluciones que violen las restricciones.

2.4. Operador de selección: torneo binario

En ambos esquemas (generacional y estacionario) se usa el **torneo binario**: se eligen dos individuos al azar de la población y se devuelve el de mayor fitness.

Algorithm 1: Torneo binario

Entrada: población P
Salida: individuo ganador
$i_1 \leftarrow$ índice aleatorio en $[0, |P| - 1]$;
$i_2 \leftarrow$ índice aleatorio en $[0, |P| - 1]$;
si $P[i_1].\mathit{fitness} \geq P[i_2].\mathit{fitness}$ **entonces**
  devolver $P[i_1]$
devolver $P[i_2]$

2.5. Operadores de cruce

Se emplean dos operadores para representación real, según la Ecuación (4) y (5) del guión.

Cruce Aritmético

Para cada posición i se genera un $r_i \in [0, 1]$ independiente:

$$Off_i^1 = r_i \cdot S_i^1 + (1 - r_i) \cdot S_i^2 \qquad Off_i^2 = r_i \cdot S_i^2 + (1 - r_i) \cdot S_i^1$$

Cruce BLX-α

Para cada posición i, siendo $I_i = |S_i^1 - S_i^2|$:

$$hmin_i = \text{mín}(S_i^1, S_i^2) - \alpha \cdot I_i \qquad hmax_i = \text{máx}(S_i^1, S_i^2) + \alpha \cdot I_i$$

Cada gen del hijo se genera uniformemente en $[hmin_i, hmax_i]$, con $\alpha = 0{,}3$.

Tras ambos cruces se aplica `fix()` a los hijos generados.

2.6. Operador de mutación

Con probabilidad $p_m = 0{,}1$ por individuo, se selecciona un gen aleatorio y se reemplaza por un valor uniforme en $[lo, hi]$, seguido de `fix()`. Solo cuenta como evaluación si efectivamente se muta.

3. Algoritmos Genéticos

3.1. Algoritmo Genético Generacional (AGG)

El AGG mantiene una población completa de $N_p = 100$ individuos que se reemplaza generación a generación. Incorpora elitismo para no perder la mejor solución encontrada.

Parámetros

- Tamaño de población: $N_p = 100$
- Probabilidad de cruce: $p_c = 0{,}8$
- Probabilidad de mutación: $p_m = 0{,}1$ (por individuo)
- Criterio de parada: 10 000 evaluaciones de la función objetivo

Esquema de evolución y reemplazamiento

Algorithm 2: AGG (esquema generacional con elitismo)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

Inicializar población P con N_p individuos aleatorios;
mientras $evals < maxEvals$ **hacer**
  $P' \leftarrow \emptyset$;
  para $i \leftarrow 1$ **to** $N_p/2$ ***y*** $evals < maxEvals$ **hacer**
    $p_1 \leftarrow$ `torneo(`P`)`; $p_2 \leftarrow$ `torneo(`P`)`;
    si $rand() \leq p_c$ **entonces**
      $(h_1, h_2) \leftarrow$ `cruzar(`p_1, p_2`)`;
    si no si $h_1 \leftarrow p_1$; $h_2 \leftarrow p_2$;
    entonces
      `mutar`
    h_1; `mutar(`h_2`)`;
    $P' \leftarrow P' \cup \{h_1, h_2\}$;
  `// Elitismo: conservar el mejor padre si la nueva generación es peor`
  si $mejor(P').fitness < mejor(P).fitness$ **entonces**
    peor(P') $\leftarrow$ mejor(P);
  $P \leftarrow P'$;
devolver $mejor(P)$

Se implementan dos variantes: **AGG-Arit** (cruce aritmético) y **AGG-BLX** (cruce BLX-α), siendo esa la única diferencia entre ellas.

3.2. Algoritmo Genético Estacionario (AGE)

El AGE genera solo dos descendientes por iteración y los inserta en la población reemplazando a los dos peores si son mejores que ellos. La población apenas varía en cada paso, lo que lo hace más conservador.

Parámetros

- Tamaño de población: $N_p = 100$
- Probabilidad de cruce: $p_c = 1{,}0$ (siempre se cruzan los dos padres)
- Probabilidad de mutación: $p_m = 0{,}1$ (por individuo)
- Criterio de parada: 10 000 evaluaciones de la función objetivo

Esquema de evolución y reemplazamiento

Algorithm 3: AGE (esquema estacionario)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

Inicializar población P con N_p individuos aleatorios;
mientras $evals < maxEvals$ **hacer**
  $p_1 \leftarrow$ `torneo(`P`)`; $p_2 \leftarrow$ `torneo(`P`)`;
  $(h_1, h_2) \leftarrow$ `cruzar(`p_1, p_2`)`;
  `mutar(`h_1`)`; `mutar(`h_2`)`;
  `// Reemplazamiento competitivo: cada hijo desafía al peor de la población`
  si $h_1.fitness > peor(P).fitness$ **entonces**
    peor$(P) \leftarrow h_1$;
  si $h_2.fitness > peor(P).fitness$ **entonces**
    peor$(P) \leftarrow h_2$;
devolver $mejor(P)$

Se implementan dos variantes: **AGE-Arit** y **AGE-BLX**.

4. Estructura del Código y Manual de Usuario

4.1. Estructura de directorios

```
portfolio/
  main.cpp                  # Punto de entrada, experimentos de 50 semillas
  CMakeLists.txt            # Sistema de construcción (CMake)
  boxplot.py                # Script Python para boxplots (pandas + seaborn)
```

```
LEEME                       # Manual de usuario
data/
  ibex_35.csv               # 30 empresas IBEX 35, 2015-2025
  syp_100.csv               # 97 empresas S&P 100, 2015-2025
  syp_500.csv               # 457 empresas S&P 500, 2015-2025
common/
  mh.h                      # Interfaz genérica MH<tDomain>
  problem.h                 # Interfaz genérica Problem<tDomain>
  random.hpp                # Generador pseudoaleatorio
  util.h                    # Utilidades auxiliares
inc/
  portfolioProblem.h        # Declaración del problema de portfolio
  randomSearch.h            # Algoritmo aleatorio
  greedy.h                  # Algoritmo greedy
  localsearch.h             # Búsqueda local (primero mejor)
  genetic.h                 # Algoritmos genéticos (AGG y AGE)
src/
  portfolioProblem.cpp      # Carga de datos, precómputo, fitness, fix
  randomSearch.cpp          # Implementación búsqueda aleatoria
  greedy.cpp                # Implementación greedy
  localsearch.cpp           # Implementación búsqueda local
  genetic.cpp               # Implementación AGG y AGE
```

4.2. Compilación

Desde el directorio raíz del proyecto:

```
cmake .
make
```

4.3. Ejecución

```
./main [max_evals] [n_seeds]
```

Ejemplos:

```
./main                  # valores por defecto: 10000 evaluaciones, 50 semillas
./main 5000             # 5000 evaluaciones, 50 semillas
./main 10000 30         # 10000 evaluaciones, 30 semillas
```

Imprime tablas de medias por pantalla y genera los ficheros CSV `results_ibex_35.csv`, `results_syp_100.csv`, `results_syp_500.csv`.

Las semillas utilizadas son $1, 2, \ldots,$ `n_seeds`, iguales para todos los algoritmos.

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	**Empresas** (N)	**Días train** (T)	**Días test**	**Rango** $[lo, hi]$
`ibex_35.csv`	30	$\sim$2560	254	[0,005, 0,08]
`syp_100.csv`	97	$\sim$2516	249	[0,005, 0,05]
`syp_500.csv`	457	$\sim$2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, `MAX_EVALS` = 10 000, 50 semillas (1–50).

5.2. Resultados

Cuadro 1: Resultados IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	—	—	—	1	—
Random	—	—	—	10 000	—
BL	—	—	—	—	—
AGG-Arit	—	—	—	10 000	—
AGG-BLX	—	—	—	10 000	—
AGE-Arit	—	—	—	10 000	—
AGE-BLX	—	—	—	10 000	—

Cuadro 2: Resultados S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	—	—	—	1	—
Random	—	—	—	10 000	—
BL	—	—	—	—	—
AGG-Arit	—	—	—	10 000	—
AGG-BLX	—	—	—	10 000	—
AGE-Arit	—	—	—	10 000	—
AGE-BLX	—	—	—	10 000	—

Cuadro 3: Resultados S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	—	—	—	1	—
Random	—	—	—	10 000	—
BL	—	—	—	—	—
AGG-Arit	—	—	—	10 000	—
AGG-BLX	—	—	—	10 000	—
AGE-Arit	—	—	—	10 000	—
AGE-BLX	—	—	—	10 000	—

5.3. Boxplots

Las Figuras 1–3 muestran la distribución del fitness de entrenamiento en las 50 ejecuciones para cada conjunto de datos.

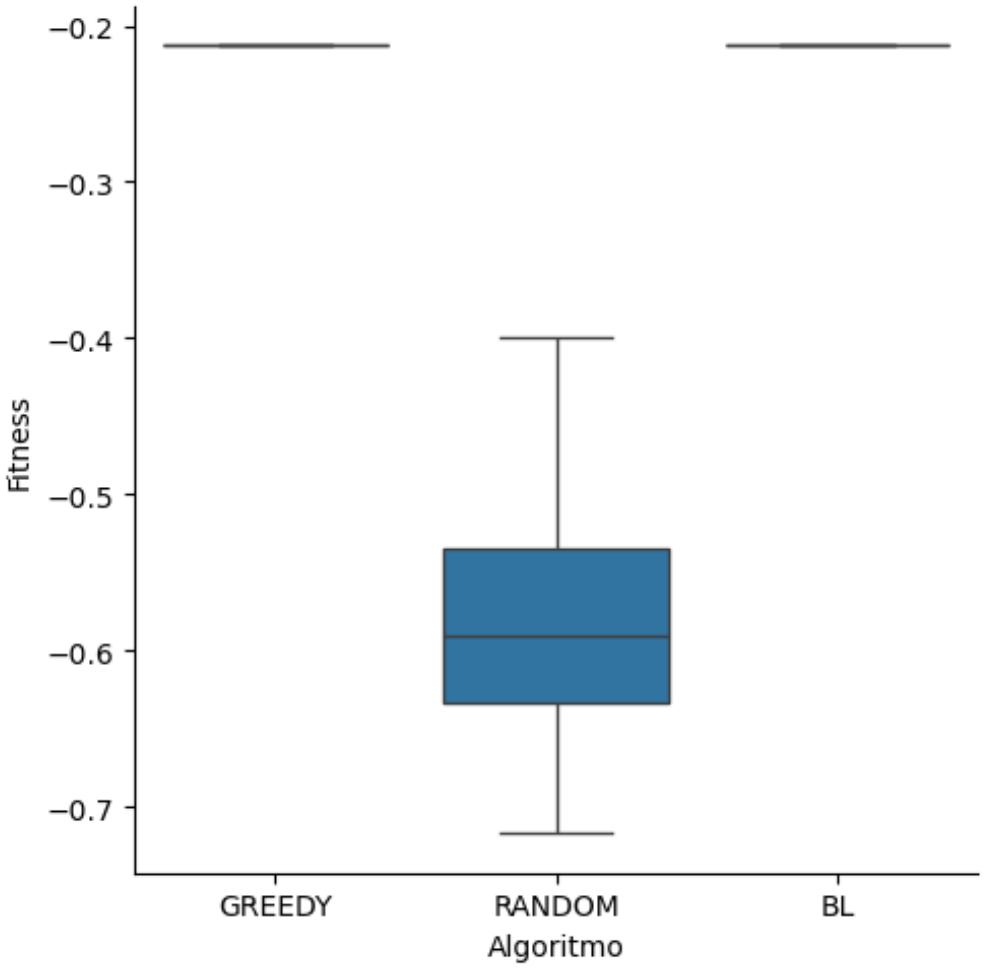


Figura 1: Distribución del fitness (2015–2024) en IBEX 35.

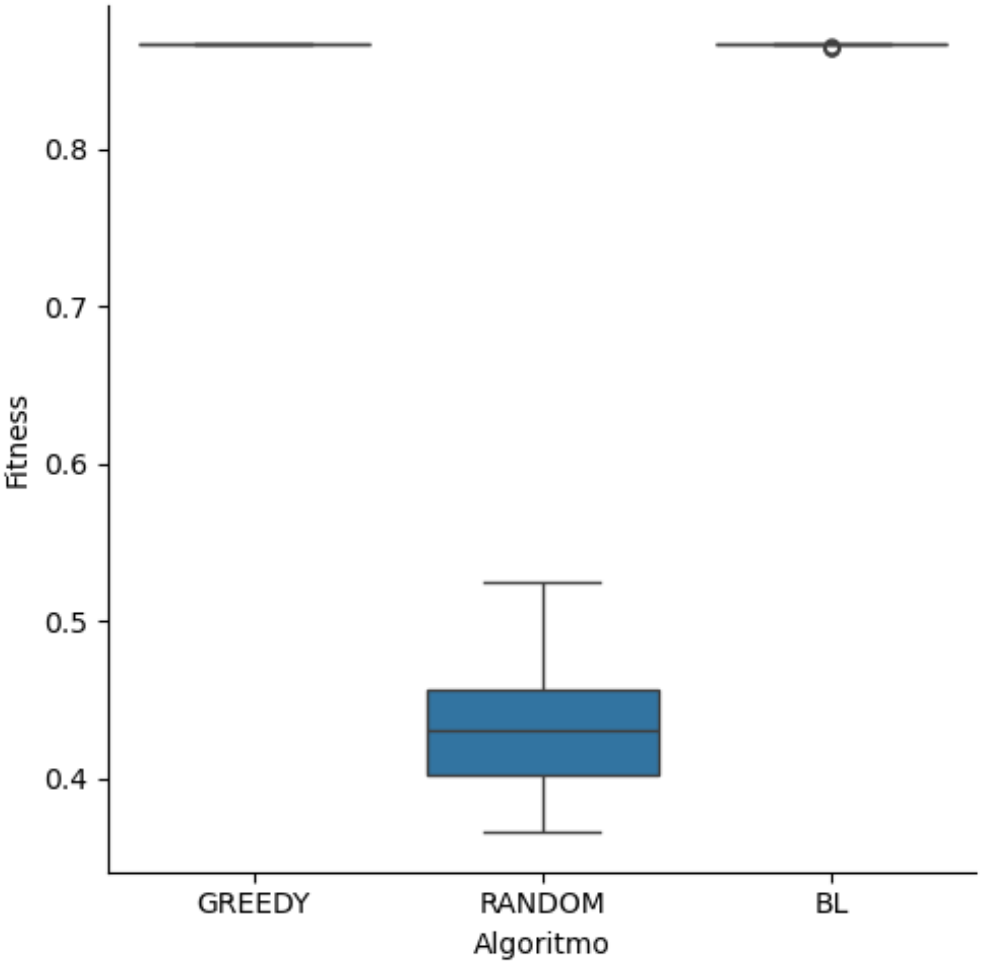


Figura 2: Distribución del fitness (2015–2024) en S&P 100.

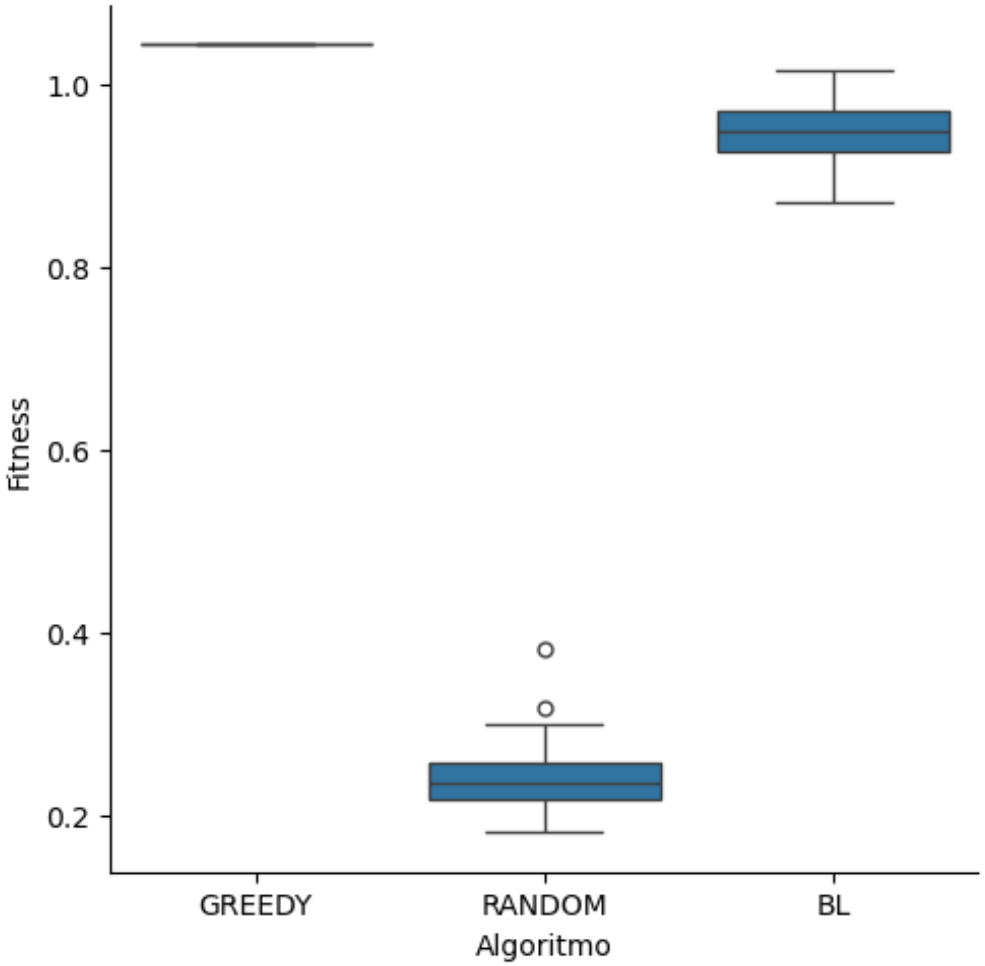


Figura 3: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. AGG-Arit vs AGG-BLX

—

6.2. AGE-Arit vs AGE-BLX

—

6.3. AGG vs AGE (comparación de esquemas)

—

6.4. AGGs/AGEs vs BL y Greedy

—

6.5. Variabilidad y coste computacional

—